

KNOWLEDGE BASE — RAG SYSTEM

Tutorial & Teaching Materials

Streaming PDF Documents via a Secure REST API

Task T-24 | Backend Engineering | Evangadi Forum Project

Covers: Input Validation • Ownership Verification • File Streaming • REST API Design

1. Overview & Purpose of This Exercise

This tutorial walks you through the complete implementation of a secure file-streaming endpoint for the Evangadi Forum Knowledge Base (RAG) system. The endpoint allows authenticated users to retrieve their privately uploaded PDF documents directly from the server so a browser can render them inline.

By the end of this exercise you will understand how a real-world Node.js / Express backend handles the full request lifecycle — from input validation, through database ownership checks, to efficient binary file delivery.

1.1 What Is the Knowledge Base (RAG) System?

RAG stands for Retrieval-Augmented Generation. In the Evangadi Forum, each learner can upload PDF documents to their private library. The system then:

- Indexes the content using semantic embeddings so passages can be found by meaning, not just keywords.
- Lets users ask AI questions that are answered exclusively from the content of their own documents.
- Provides a semantic search interface to locate relevant excerpts.
- Streams the original PDF back to the browser so the learner can read it inline.

The endpoint you are building — `GET /api/rag/documents/:documentId/file` — powers the PDF preview panel visible on the right-hand side of the Knowledge Base page.

1.2 Learning Objectives

After completing this exercise you should be able to:

1. Understand why input validation belongs in its own dedicated layer and how express-validator enforces it.
2. Explain the concept of resource ownership and implement an ownership check before serving sensitive data.
3. Use Node's `path.resolve()` to safely translate relative database paths to absolute file-system paths.
4. Stream a binary file back to an HTTP client using Express's `res.sendFile()` and set the correct Content-Type header.
5. Handle errors gracefully at every stage of a controller and return appropriate HTTP status codes.
6. Structure a feature across validation, controller, and routing files following the separation-of-concerns principle.

Why does this endpoint matter?

Without this endpoint the Preview tab in the right-hand panel cannot display the PDF. It is also the most security-sensitive read endpoint in the RAG module: if ownership is not verified, any authenticated user could read another user's private documents simply by guessing a numeric ID. The lesson here applies to every resource-level access check in any web application you will ever build.

2. Key Concepts & Background Knowledge

2.1 The Three-Layer Architecture

The implementation deliberately spreads responsibility across three files. This pattern is called Separation of Concerns and is a cornerstone of maintainable back-end code.

File	Responsibility	Analogy
<code>rag.validation.js</code>	Input validation	Airport security: check the ticket before boarding
<code>rag.controller.js</code>	Business logic	Cabin crew: decide what happens on the flight
<code>rag.routes.js</code>	HTTP wiring	Gate agent: direct passengers to the right flight

2.2 What is express-validator?

express-validator is a Node.js library that lets you declare rules about incoming request data — path parameters, query strings, and body fields — before your controller code ever runs. If the data violates a rule the library collects an error and your controller can read it with `validationResult(req)`.

The core functions used in this task are:

- `param("documentId")` — targets the `:documentId` segment of the URL path.
- `.notEmpty()` — rejects undefined or empty string values.
- `.isInt({ gt: 0 })` — ensures the value is a whole number greater than zero.
- `.toInt()` — coerces the string `"42"` into the Number `42` before the controller reads it.

Remember

Express always receives URL segments as strings. Without `.toInt()` you would compare the string `"42"` to a numeric database column, which can cause silent type-mismatch bugs in some database drivers.

2.3 Resource Ownership — Why It Matters

Every document row in the database has a `user_id` column that records who uploaded it. When a request arrives for document 42, the server must answer one question before doing anything else:

"Does document 42 belong to the user making this request?"

If the server skips this check and simply looks up the file path for any documentId, a malicious user could enumerate IDs (1, 2, 3 ...) and download every PDF ever uploaded by every user. This class of vulnerability is called an Insecure Direct Object Reference (IDOR) and appears regularly in the OWASP Top 10 list of critical web security risks.

The `assertOwnedDocument` helper encapsulates the ownership check. It queries the database for a row matching both the documentId AND the authenticated user's id. If no row is found it returns null (or throws), and the controller immediately returns a 404 response.

Security Note

Returning 404 (Not Found) rather than 403 (Forbidden) when ownership fails is intentional. A 403 would confirm that the document exists but the caller lacks access — information an attacker could exploit. A 404 reveals nothing about whether the document exists at all.

2.4 Absolute vs Relative File Paths

The `storage_path` column in the database stores a relative path such as:

```
uploads/rag/42/react-routing-notes.pdf
```

This relative path is interpreted relative to the current working directory of the Node process (the project root). `path.resolve()` converts it to an absolute path that the operating system can locate directly:

```
path.resolve("uploads/rag/42/react-routing-notes.pdf")  
// => "/home/ubuntu/evangadi-forum/uploads/rag/42/react-routing-notes.pdf"
```

Using an absolute path with `res.sendFile()` is a security best practice because it eliminates any ambiguity about which directory the file lives in.

2.5 `res.sendFile()` — How File Streaming Works

`res.sendFile()` is Express's built-in method for serving files. Compared to reading the file into memory and then writing the buffer to the response, `sendFile`:

- Streams the file directly from disk using Node's stream piping — memory usage stays constant regardless of file size.
- Sets the Content-Type header automatically based on the file extension (.pdf → application/pdf).
- Adds ETag and Last-Modified headers so browsers can cache the file correctly.
- Accepts an optional callback that fires when streaming finishes or fails.

The callback pattern is important: if an error occurs after headers have already been sent (because streaming started), you cannot send another response. The guard `!res.headersSent` prevents a double-response crash.

3. Step-by-Step Implementation Guide

Step 1 — Add documentIdParamValidation in rag.validation.js

Open rag.validation.js and add the following validator alongside your existing ones. This is the very first line of defence: if the caller passes ?documentId=abc or omits the segment entirely, the request is rejected before it touches any database or file system.

```
// rag.validation.js
const { param } = require("express-validator");

const documentIdParamValidation = [
  param("documentId")
    .notEmpty()
    .withMessage("documentId is required")
    .isInt({ gt: 0 })
    .withMessage("documentId must be a positive integer")
    .toInt(),
];

module.exports = {
  // ...existing validators,
  documentIdParamValidation,
};
```

What each chain method does

Method	Effect
param("documentId")	Selects the :documentId URL segment as the target
.notEmpty()	Fails if the value is undefined, null, or empty string
.isInt({ gt: 0 })	Fails if the value is not a whole number greater than 0
.withMessage(...)	Sets the human-readable error returned to the caller
.toInt()	Converts the string "5" into the Number 5

Step 2 — Implement getDocumentFileController in rag.controller.js

The controller is the orchestrator. It calls validate → verify → resolve → stream in sequence and handles every failure case with an appropriate HTTP status code.

```
// rag.controller.js
```

```

const path = require("path");
const { validationResult } = require("express-validator");
const { assertOwnedDocument } = require("../rag.repository");

const getDocumentFileController = async (req, res) => {

  // — Step 1: Check validation results —————
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }

  const { documentId } = req.params; // already a Number via .toInt()
  const userId = req.user.id;       // set by auth middleware

  try {
    // — Step 2: Verify ownership —————
    const document = await assertOwnedDocument(documentId, userId);

    if (!document) {
      return res.status(404).json({ message: "Document not found." });
    }

    // — Step 3: Resolve absolute file path —————
    const absolutePath = path.resolve(document.storage_path);

    // — Step 4: Stream the file —————
    return res.sendFile(
      absolutePath,
      { headers: { "Content-Type": "application/pdf" } },
      (err) => {
        if (err && !res.headersSent) {
          return res.status(500).json({ message: "Could not stream the file." });
        }
      }
    );
  } catch (error) {
    if (error.status === 404) {
      return res.status(404).json({ message: "Document not found." });
    }
    return res.status(500).json({ message: "Internal server error." });
  }
};

module.exports = { getDocumentFileController };

```

Step 3 — Register the Route in rag.routes.js

The route file wires the three pieces together: authentication middleware, the validator array, and the controller function. Express executes them left-to-right in the order they are listed.

```
// rag.routes.js
const express = require("express");
const router = express.Router();

const { authenticate } = require("../middleware/auth");
const { documentIdParamValidation } = require("../rag.validation");
const { getDocumentFileController } = require("../rag.controller");

router.use(authenticate); // all RAG routes require a Bearer token

// ... existing routes ...

// NEW: Stream PDF file
router.get(
  "/documents/:documentId/file",
  documentIdParamValidation, // 1. validate :documentId
  getDocumentFileController // 2. verify → resolve → stream
);

module.exports = router;
```

Step 4 — Run Git Workflow

Always work on a feature branch. Never commit directly to main.

```
git checkout main
git pull origin main
git checkout -b feature/T-24-rag-documents

# Make your changes to the three files

git add .
git commit -m "[T-24] Implement GET /api/rag/documents/:documentId/file"
git push origin feature/T-24-rag-documents
```

4. Full Request Lifecycle Walkthrough

The diagram below traces a successful request and every possible failure path from the moment the browser sends the HTTP request to the moment it receives the PDF bytes or an error response.

Lifecycle Overview

1. Browser → GET /api/rag/documents/42/file (Authorization: Bearer <token>)
2. authenticate middleware → verify token, attach req.user
3. documentIdParamValidation → check "42" is a positive integer, coerce to Number
4. validationResult(req) → if errors, return HTTP 400
5. assertOwnedDocument(42, userId) → query DB for document WHERE id=42 AND user_id=userId
6. if not found → return HTTP 404
7. path.resolve(document.storage_path) → build absolute path on disk
8. res.sendFile(absolutePath) → stream bytes with Content-Type: application/pdf
9. if streaming error → return HTTP 500 (only if headers not yet sent)

4.1 HTTP Status Codes Used

Status	Meaning in This Endpoint
200 OK	File streamed successfully (set automatically by <code>res.sendFile</code>)
400 Bad Request	<code>documentId</code> failed validation (not a positive integer)
401 Unauthorized	No Bearer token or invalid token (set by auth middleware)
404 Not Found	Document does not exist or belongs to a different user
500 Internal Server Error	Unexpected DB or file-system error

4.2 Why Each Failure Returns 404 Instead of 403

When a user requests a document that exists but belongs to someone else, the server returns 404 Not Found rather than 403 Forbidden. This is a deliberate security decision:

- 403 Forbidden would confirm to an attacker that the document exists — they just cannot access it.
- 404 Not Found reveals no information. From the caller's perspective, the document simply does not exist.
- This technique is called security through ambiguity at the resource level and is widely adopted in production APIs.

5. Function Deep-Dives

5.1 validationResult(req)

This function is imported from `express-validator`. It inspects the request object for any validation errors that the middleware chain accumulated and returns a `Result` object.

```
const errors = validationResult(req);

if (!errors.isEmpty()) {
  // errors.array() returns something like:
  // [{ type: 'field', msg: 'documentId must be a positive integer',
  //    path: 'documentId', location: 'params', value: '-1' }]
  return res.status(400).json({ errors: errors.array() });
}
```

- `errors.isEmpty()` returns `true` when there are no violations — the happy path.
- `errors.array()` serialises every collected error into a plain array for easy JSON serialisation.
- Returning early with `return` prevents the rest of the controller from running on invalid data.

5.2 assertOwnedDocument(documentId, userId)

This repository helper runs a database query equivalent to:

```
SELECT * FROM rag_documents
WHERE id = $1
      AND user_id = $2
LIMIT 1;
```

The two-column `WHERE` clause is critical. Without the `AND user_id = $2` clause the function would return any document matching the ID, regardless of who owns it. With both conditions only a row that is simultaneously the right document AND owned by the current user is returned.

The function either returns the document row object (with its `storage_path` column) or returns `null` / throws, triggering the 404 branch in the controller.

5.3 path.resolve()

`path.resolve()` is a Node.js built-in that converts one or more path segments into a single absolute path. When given a relative path it prepends the current working directory (`process.cwd()`):

```
// process.cwd() = "/home/ubuntu/evangadi-forum"
```

```
path.resolve("uploads/rag/42/notes.pdf")
// => "/home/ubuntu/evangadi-forum/uploads/rag/42/notes.pdf"

// It also normalises double slashes and . / .. segments:
path.resolve("uploads/../uploads/rag/42/notes.pdf")
// => "/home/ubuntu/evangadi-forum/uploads/rag/42/notes.pdf"
```

`res.sendFile()` requires an absolute path by default. Passing a relative path without a root option causes Express to throw an error.

5.4 `res.sendFile()`

`res.sendFile(path, options, callback)` is the Express method that reads and streams a file from disk directly to the HTTP response. The full signature used here:

```
res.sendFile(
  absolutePath,
  { headers: { "Content-Type": "application/pdf" } },
  (err) => {
    if (err && !res.headersSent) {
      return res.status(500).json({ message: "Could not stream the file." });
    }
  }
);
```

Argument	Purpose
<code>absolutePath</code>	The full disk path to the PDF file
<code>headers: { Content-Type }</code>	Tells the browser the file is a PDF so it renders it inline rather than downloading
<code>callback (err)</code>	Called when streaming completes or fails; guards against double-response with <code>!res.headersSent</code>

6. Testing the Endpoint

6.1 Happy Path — Using Postman

7. Open Postman and create a new GET request.
8. Set the URL to: `http://localhost:5000/api/rag/documents/1/file`
9. Add the Authorization header: `Bearer <your_jwt_token>`
10. Click Send. The response body should display a PDF preview in Postman.
11. Check the response headers: Content-Type should be `application/pdf`.

6.2 Error Cases to Test

Test Case	Expected Result
No token	401 Unauthorized from auth middleware
documentId = abc	400 Bad Request — fails <code>isInt</code> validation
documentId = -5	400 Bad Request — fails <code>gt: 0</code> validation
documentId = 9999 (non-existent)	404 Not Found — <code>assertOwnedDocument</code> returns null
documentId belongs to another user	404 Not Found — ownership check fails
Valid owned document	200 OK with PDF bytes in body

6.3 Postman cURL Equivalent

You can also test directly from a terminal using cURL:

```
curl -X GET http://localhost:5000/api/rag/documents/1/file \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  --output preview.pdf

# Then open preview.pdf in any PDF viewer to confirm the file is correct.
```

7. Common Mistakes & How to Avoid Them

Mistake	Consequence / Fix
Forgetting .toInt()	<code>documentId</code> arrives as the string "42" and a strict DB equality check with the integer column may silently return no rows
Skipping ownership check	Any authenticated user can read any document – critical IDOR vulnerability
Passing relative path to <code>sendFile</code>	Express throws <code>TypeError: path must be absolute</code> unless you provide a <code>root</code> option
Not checking <code>!res.headersSent</code>	Calling <code>res.status()</code> after headers are sent causes a crash: "Cannot set headers after they are sent"
Returning 403 for missing documents	Reveals to an attacker that the document exists – always use 404 for ownership failures
No try/catch in controller	Uncaught DB errors crash the Express process instead of returning a clean 500

8. Lessons Learned & Best Practices

8.1 Always Validate at the Boundary

Data from the network is untrusted by definition. Validate every piece of incoming data — path params, query strings, request bodies, and headers — before any business logic runs. `express-validator` makes this declarative and composable.

8.2 Ownership Checks Are Not Optional

Any endpoint that serves user-specific data must verify that the authenticated user actually owns the resource they are requesting. Delegating this to a named helper function (`assertOwnedDocument`) makes the pattern explicit, reusable, and easy to audit in code review.

8.3 Return Errors That Leak the Minimum Information

Distinguish between what the client needs to know (there was a problem) and what the server needs to know (the precise error). Return 404 for ownership failures, not 403. Log the detailed error server-side only.

8.4 Separate Concerns Across Files

Validation logic does not belong in the controller. Routing config does not belong in the controller either. Each file has exactly one job — this makes the codebase easier to test, easier to change, and easier for new engineers to understand.

8.5 Streaming Over Buffering

`res.sendFile()` streams directly from disk. Never do this:

```
// BAD — loads the entire file into RAM
const buffer = fs.readFileSync(absolutePath);
res.send(buffer);
```

For large PDFs the above approach will consume hundreds of megabytes of heap memory per concurrent request. `res.sendFile()` keeps memory usage at a flat, predictable level regardless of file size.

8.6 Feature Branches Are Non-Negotiable

Every task — no matter how small — gets its own branch. This keeps main stable, makes code review focused, and creates a clean audit trail in the Git history. The commit message convention [T-24] ties the commit directly to the task tracker entry.

End of Tutorial — Evangadi Forum | RAG Knowledge Base | Task T-24